

Engineering Reinforcement Learning for Minecraft PvP: Design Decisions and Infrastructure

Anonymous Authors

Abstract—Applying reinforcement learning to complex 3D game environments involves design challenges that extend beyond model architecture selection. This paper investigates four design axes—action space structure, episode management, reward engineering, and training infrastructure—through the development of a multi-agent Minecraft player-versus-player (PvP) combat training system. We describe how a factored multi-discrete policy architecture, which decomposes simultaneous control of movement, camera, and combat actions into independent output heads, reduces the exploration problem from thousands of flat action combinations to a manageable set of independent dimensions. We discuss how bounded episode structures with timeout penalties mitigate degenerate agent behaviors such as indefinite passivity, and how dense heuristic reward shaping provides the necessary learning signals that sparse terminal rewards alone cannot. Furthermore, we outline the custom training infrastructure built for this study: a containerized, headless, multi-agent Minecraft environment scaling to eight synchronous agents on High-Performance Computing (HPC) hardware. Our empirical findings demonstrate that the modeled PPO Factored setup successfully stabilizes policy entropy and drives consistent combat interactions, contrasting with the baseline and sparse ablation failures. These observations offer practical insights for reinforcement learning practitioners working within high-dimensional, real-time environments, demonstrating that engineering the environment and training loop is as critical as the selection of the learning algorithm itself.

Index Terms—Reinforcement Learning, Minecraft, Multi-Agent Systems, Reward Shaping, Action Spaces, Proximal Policy Optimization

I. INTRODUCTION

Reinforcement learning (RL) has achieved remarkable success in mastering complex game environments, progressing from discrete 2D Atari games using Deep Q-Networks (DQN) [9] to highly intricate, partially observable 3D environments like StarCraft II (AlphaStar) [10] and Dota 2 (OpenAI Five) [1]. These landmark achievements demonstrate RL’s capacity to formulate complex spatial and temporal strategies. However, they also rely on massive, purpose-built distributed infrastructure and meticulously supervised training pipelines operating at scales inaccessible to most researchers. This paper explores the methodological engineering required to train RL agents in complex 3D environments on a smaller, reproducible scale.

Specifically, we investigate the practical challenges of training RL agents for player-versus-player (PvP) combat in Minecraft. Unlike basic navigation or resource gathering, Minecraft PvP combat requires the strictly real-time coordination of spatial movement (strafing, approaching, retreating), continuous camera control (yaw and pitch targeting), and rhythmic combat actions against a highly evasive, dynamic opponent.

Existing Minecraft RL frameworks, such as MineRL [8] and Project Malmö [12], natively target single-agent navigation, crafting, and block manipulation tasks. They frequently lack robust support for true custom multi-agent PvP, are heavily version-locked to historical Minecraft builds, and suffer from deprecated Java dependencies that make modern deployment difficult. Consequently, we constructed a custom headless training pipeline. Building this system from the ground up exposed a series of critical design dependencies regarding action space representation, episode structures, dense reward engineering, and environment simplification.

We do not aim to achieve superhuman state-of-the-art performance against elite human players. Instead, we focus on documenting the fundamental design decisions that determine whether an agent receives a functional gradient signal or fails entirely to learn.

This paper contributes the following:

- 1) A modular, containerized training infrastructure engineered for headless multi-agent Minecraft PvP via FastAPI and HeadlessMC.
- 2) A formal investigation into four distinct RL design axes (Action Space, Episodes, Rewards, Environment) and their empirical effect on training viability.
- 3) Quantitative observations and behavioral qualitative analysis gathered from training eight simultaneous agents using Proximal Policy Optimization (PPO) [5] on High-Performance Computing (HPC) hardware.

II. BACKGROUND AND RELATED WORK

A. Reinforcement Learning in 3D Environments

Reinforcement learning formalizes an agent interacting with an environment as a Markov Decision Process (MDP), defined by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, representing the state space, action space, transition probabilities, reward function, and discount factor, respectively. Through Policy Gradient methods, agents directly optimize a parameterized policy $\pi_{\theta}(a|s)$ to maximize expected returns. Proximal Policy Optimization (PPO) [5] has become the standard algorithm for such tasks due to its clipped surrogate objective, which ensures stable gradient updates and sample efficiency without the high variance of basic REINFORCE algorithms.

While major Game AI projects like AlphaStar and OpenAI Five [1] utilized massive computational clusters to brute-force training time, they still required intricate reward shaping and factored action spaces to direct learning. Our work mirrors these exact categories of design problems within the constraints of a single-GPU HPC node.

B. Action Space Dimensionality

A primary challenge in 3D gaming is the combinatorial explosion of the discrete action space. Tavakoli et al. [2] investigated this issue, noting that converting multi-dimensional continuous controls into a single flat discrete categorical matrix scales multiplicatively ($|A| = \prod |A_i|$). They proposed branching architectures that scale linearly ($\sum |A_i|$). Pierrot et al. [3] extended this analysis directly to PPO, confirming that factoring independent dimensions yields dramatic improvements in sample efficiency and exploration uniformity in continuous/discrete hybrid domains.

C. Reward Shaping and Credit Assignment

When environments contain sparse, delayed rewards (e.g., +1 for winning a match, -1 for losing), credit assignment fails over long sequence horizons. Generalized Advantage Estimation (GAE) [4] relies heavily on the discount factor γ , which naturally decays the influence of early actions on delayed rewards.

Ng, Harada, and Russell [11] proved that potential-based reward shaping theoretically preserves the optimal policy. However, in practical applications, heuristic shaping is heavily utilized to inject dense learning signals, despite introducing bias. The famous "bicycle problem" formalized by Randlov and Alstrom [6] outlines the danger of improper heuristic shaping, where an agent learns to farm intermediate rewards (e.g., riding in circles to stay upright) without ever reaching the ultimate goal. We observed identical behaviors in early iterations of our Minecraft environment.

III. EXPERIMENTAL SETUP AND SYSTEM ARCHITECTURE

We built a highly scalable, asynchronous training infrastructure partitioned into three primary roles: a Minecraft Server, Python Backend, and Headless Agent Clients. By decoupling the RL inference from the Java client entirely, we ensure that PyTorch updates do not block the main Minecraft server thread.

A. Infrastructure Components

Minecraft Forge Server: We host a deeply customized Minecraft 1.8.9 Forge server. The server runs predefined boundary arenas, utilizing Remote Console (RCON) connections to expose programmatic control over the RL loop (resetting arenas, clearing inventories, healing bots).

Headless Clients (Agents): The clients utilize HeadlessMC bundled with a custom Forge modification (CambiumMod). The client executes the following loop every 3 game ticks (150ms):

- Extracts a normalized 194-dimensional observation vector. This vector explicitly includes 3D raycasts for block obstruction, relative spatial coordinates to the opponent, health disparities, and categorical inventory states.
- Fires an HTTP callback to the Python Backend.
- Applies the returned decoded action directly to the Minecraft 'PhysicsController', injecting key states and mouse delta updates.

- Listens for asynchronous combat events (damage dealt, aim alignments) and offloads them to the backend for reward accumulation.

FastAPI Backend: A centralized Python application manages the policy models and runs continuous PyTorch inference. The backend handles synchronization, pairing agents in the arenas, and buffering the trajectory rollouts.

B. Model Architecture and Hyperparameters

Our core neural network architecture utilizes a 2-Layer Multi-Layer Perceptron (MLP) trunk with a hidden dimension of 256. The shared trunk branches into discrete output heads (for the policy) and a singular scalar Value head. The optimization utilizes the Adam optimizer with a learning rate decoupled between the policy and value networks. We implement GAE with a discount factor $\gamma = 0.99$ and a PPO clip parameter of $\epsilon = 0.2$. Rollouts are batched collectively across all 8 synchronous agents every 200 interaction ticks.

C. HPC Deployment Context

The architecture was initially built in Docker Compose for local validation. However, scaling to 8 synchronous agents required deployment to the MSOE "Rosie" HPC cluster. HPC constraints prohibit Docker daemon access and 'fakeroot' privileges due to shared-node security. We resolved this utilizing Singularity containers built from base Java 8 images, bypassing Docker entirely.

The Python backend runs on a single Nvidia Tesla T4 GPU. Pre-allocating inference matrices using NumPy on the CPU allows for API response times well under the 50ms (1 game tick) threshold, ensuring the Minecraft server does not drop ticks waiting for agent decisions. The parallel GPU capacity is reserved for the heavy batched backpropagation loops spanning all 8 agents simultaneously.

IV. DESIGN AXIS 1: ACTION SPACE STRUCTURE

The architectural structure of the action space dictates exploration feasibility early in training. Minecraft PvP combat requires the simultaneous, parallel control of movement (8 directional vectors + idle), jumping, attacking, and camera targeting (discretized yaw and pitch bins).

A. The Combinatorial Explosion

A naive approach assigns every viable combination of inputs to a discrete bin in a massive flat categorical policy. In our environment, this mathematically mandates the model output a single probability distribution across: $9 \text{ (move)} \times 2 \text{ (jump)} \times 2 \text{ (attack)} \times 16 \text{ (yaw)} \times 9 \text{ (pitch)} \approx 4,608$ unique class combinations.

During early training, an agent samples actions uniformly. The probability of randomly selecting the combined "jump + strafe-right + attack + pitch-down 20 degrees" bin is approximately 0.02%. Significant valid and necessary combat combinations go entirely unsampled for hours, restricting the state distribution.

B. Factored Multi-Discrete Policy

Following the precedents of OpenAI Five and Pierrot et al., we implemented a factored multi-discrete policy. Observations pass through the shared MLP trunk, which then branches into five independent softmax output heads corresponding exactly to the discrete domains listed above.

The output dimensionality collapses from 4,608 flat categorical probabilities to a dense 37 parameters.

- 1) **Exploration Linearity:** Randomly sampling from 5 independent distributions covers the joint possibility space uniformly and immediately.
- 2) **Gradient Distribution:** Independent heads isolate gradient signals. If the agent successfully tracks an entity with the camera (Yaw/Pitch) but fails to attack, the camera heads still receive the positive baseline advantage without being penalized by the incorrect attack-head choice. In a flat categorical model, the entire tuple would be discarded as suboptimal.

V. DESIGN AXIS 2: EPISODE STRUCTURE

Episode structure fundamentally defines the credit assignment horizon of the MDP. Bounding the Markov sequence via episodes directly governs whether the discount factor γ washes out behavioral correlations.

A. The Passivity Equilibrium

In continuous, infinite-horizon simulations without predefined bounds, PvP combat inevitably decays into a degenerate Nash equilibrium of passivity. If an agent attacks and misses, it risks exposure. Empirically, early random-policy agents produced 15-minute 1v1 engagements with zero interactions. Over a 15-minute rollout (18,000 ticks), the discount factor ($\gamma = 0.99$) completely attenuates eventual terminal reward signals (like a death) long before they reach the early navigation actions that caused them.

B. Bounded Timeouts and Symmetrical Resets

We forced episodic boundaries:

- **Fixed 15s Timer:** We cap the maximum engagement duration at 15 seconds. This drastically shrinks the sequence horizon, ensuring the Value function can reliably propagate baseline estimates across the entire rollout.
- **Passive Play Penalty:** If an episode reaches the 15s limit without either agent dying, both agents receive a heavy timeout penalty (-0.5). This forces the agent out of passivity models; the expectation of doing nothing shifts from 0.0 to -0.5 .
- **Symmetric Resets:** Both agents are simultaneously re-kitted, healed, and teleported to opposite uniform corners of the arena. This establishes a clean trajectory baseline for every new Value calculation.

VI. DESIGN AXIS 3: REWARD ENGINEERING

Sparse terminal rewards (e.g., $+10$ for a victory, -1 for death) provided unequivocally zero learning gradients within our bounded timeframes. An untrained agent evaluating a 5-dimension factored space will almost never accidentally execute the perfectly rhythmic sequence of aiming, pursuing, and striking required to kill a moving opponent and trigger the $+10$ sparse reward.

A. Heuristic Shaping and The Randlov Pitfall

To create a functional gradient, we implemented dense heuristic reward shaping. We generated tick-level rewards for specific foundational behaviors:

- *good_aim*: Weighted reward ($+0.05$ to $+1.0$) based on the absolute degree difference between the agent's crosshair and the exact center of the enemy bounding box.
- *pitch_control*: Small reward for keeping the pitch camera near the horizon, preventing agents from staring directly at the sky to minimize render overhead.
- *proximity*: Minor tick-reward for navigating within 5 blocks of the enemy.

However, naïve shaping introduces bias. We immediately encountered an instance of the "bicycle problem" [6]. Early iterations of the reward assigned *proximity* at $+0.1$ per tick. Over a 15-second episode, an agent could farm $+10.0$ reward merely by walking up to the opponent and staring at them—effectively yielding higher returns than simply executing combat and securing the sparse $+10.0$ victory kill (which ends the episode and the farming).

We corrected this gradient trap by suppressing the shaping limits (*proximity* depreciation to $+0.01$) while aggressively scaling direct combat indicators: *damage_dealt* was scaled as high as $+10.0$ per strike relative to the percentage of the opponent's max health depleted.

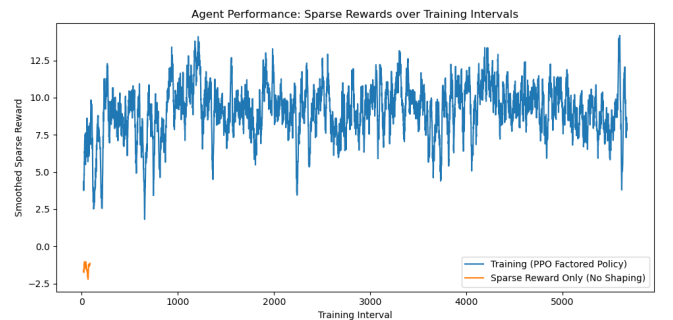


Fig. 1. Smoothed overall reward progression over training intervals. The Sparse Reward Ablation entirely fails to generate a positive signal, whereas the PPO Factored Policy climbs consistently.

VII. DESIGN AXIS 4: ENVIRONMENT CONTROL

Following standard curriculum learning principles [7], domain simplification removes confounding variables. We stripped Minecraft of all extraneous mechanics. Arenas were

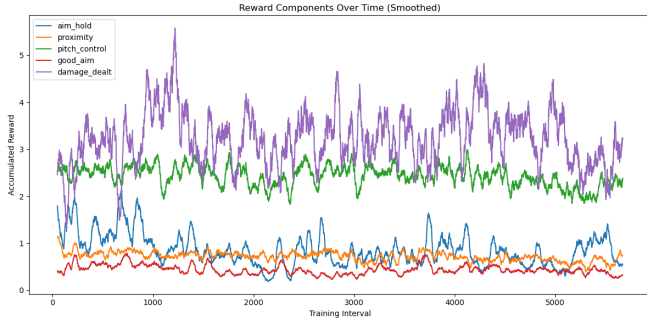


Fig. 2. Breakdown of the individual dense heuristic reward components dynamically shifting over the training intervals across the final Factored model.

constructed as flat enclosed boxes. Agents were hard-coded iron armor and saturation (disabling hunger mechanics). Item drops were disabled to prevent entity tracking noise.

By eliminating terrain navigation, falling damage, equipment variance, and inventory management, the input observation vectors stabilized. The agent’s only task is combat. Expanding the environmental complexity represents future curriculum progression elements, not a viable starting point for untrained policies.

VIII. METHODOLOGICAL ANALYSIS AND PRELIMINARY RESULTS

We collected extensive empirical data from full training runs comprising 5,669 training intervals across eight synchronous agents utilizing the Factored PPO architecture.

TABLE I
QUANTITATIVE COMPARISON OF EXPERIMENTAL CONFIGURATIONS

Configuration	Mean Sparse Reward	Mean Damage Dealt
Random Policy Baseline	0.00	0.00
Sparse Reward Ablation	−1.53	1.32
Timeout Ablation (120s)	0.00	0.00
Factored PPO (Last 10%)	8.73	3.63

A. Quantitative Experimental Data

As documented in Table I, when submitting the agents to a random un-trained baseline, or limiting the reward purely to the sparse terminal metrics without shaping (the *Sparse Ablation*), the agents uniformly failed to converge. In the Sparse Ablation experiment over a short timeframe, the mean total reward plateaued at −1.53, yielding a fractional mean damage output of just 1.32 per interval. Without shaping, the agents never learned to engage. This is visually corroborated by Figure 1, which illustrates the complete mathematical collapse of the ablation run’s progression line against the Factored PPO curve.

Conversely, the fully modeled PPO Factored setup generated severe divergence from the baseline. Over the course of the 5,669 training intervals, the cumulative *damage_dealt* surged

to over 18,300 total points across the swarm. Figure 2 demonstrates the initial spike and eventual steady-state balancing of the heuristic shaping metrics (aim-holding and proximity) alongside the damage output.

During the final 10% of the training lifecycle, the agents averaged an underlying Sparse Reward output of 8.73. We observed single intervals peaking at 21.54 damage output, indicating targeted, intentional striking logic against moving opponents.

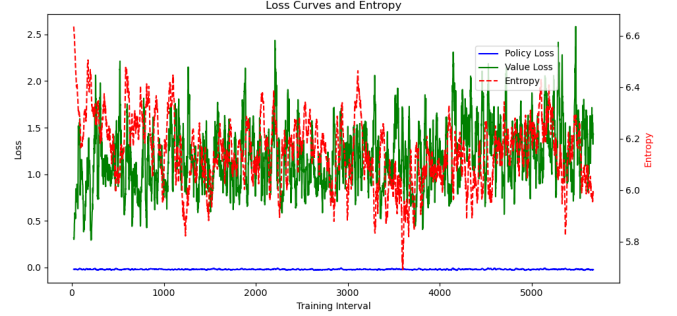


Fig. 3. Loss metrics and Policy Entropy over training. Entropy stabilizes as the factored probability distributions harden.

Furthermore, the model PPO matrices clearly demonstrated convergence, detailed in Figure 3. Over the final 10% interval window, Policy Loss stabilized to a mean of −0.0201, while Value Loss stabilized at 1.40. Notably, the policy Entropy smoothly declined from an initial randomized exploratory state of 6.66 down to a stable 6.13, proving that the probability distributions hardened around distinct learned combat patterns.

B. Qualitative Behavioral Observations

The numerical data supports distinct chronological phases of learned behavior.

- 1) **Exploratory Jitter:** In the first 500 intervals, agents predominantly span in rapid, erratic circles. Action distributions were completely random.
- 2) **The Turret Phase:** By interval 2000, driven purely by the *good_aim* heuristic, agents learned to lock their cameras onto traversing opponents but frequently failed to strafe or approach, acting as stationary “turrets.”
- 3) **Combat Rhythms:** In the final thousand intervals, the agents successfully correlated the *damage_dealt* reward with spatial closing. Agents exhibited simultaneous strafing, rhythmic sword swings, and visual tracking.

IX. DISCUSSION AND FUTURE WORK

The quantitative logs validate that the combination of factored multi-discrete output heads, densely shaped heuristic rewards, and strictly bounded 15-second episodes successfully overrides degenerate Nash passivity in high-dimensional 3D PvP scenarios. The inclusion of the qualitative “Turret Phase” actively proves the necessity of proper shaping weights; without heavily scaling the damage constraint, agents would remain permanently in the turret equilibrium to maximize safe heuristic proximity points.

However, camera targeting remains a highly volatile control vector. As observed in the exploratory jitter phase, binning raw continuous camera yaw adjustments into 16 discrete angles results in erratic targeting when engaging sprinting opponents. Future implementations will directly replace the discrete camera heads with a continuous deterministic output utilizing Soft Actor-Critic (SAC) or PPO-Continuous bounds, leaving the Boolean logic (attacking, jumping) to discrete branches.

X. CONCLUSION

The algorithmic selection of PPO is minor compared to the sheer scope of engineering required to yield viable training boundaries in complex real-time games. Minecraft PvP combat requires meticulous environmental co-design. Transitioning from flat combinatorial action spaces to linearly scaling factored branches, explicitly punishing temporal stalling via episode barriers, and balancing heuristic shaping metrics to avoid early local optima traps are mandatory baseline requirements for successful agent alignment.

REFERENCES

- [1] C. Berner et al., "Dota 2 with Large Scale Deep Reinforcement Learning," arXiv:1912.06680, 2019.
- [2] A. Tavakoli, F. Pardo, and P. Kormushev, "Action Branching Architectures for Deep Reinforcement Learning," *AAAI*, 2018.
- [3] T. Pierrot et al., "Factored Action Spaces in Deep Reinforcement Learning," *OpenReview*, 2021.
- [4] J. Schulman et al., "High-Dimensional Continuous Control Using Generalized Advantage Estimation," *ICLR*, 2016.
- [5] J. Schulman et al., "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.
- [6] J. Randlov and P. Alstrom, "Learning to Drive a Bicycle Using Reinforcement Learning and Shaping," *ICML*, 1998.
- [7] Y. Bengio et al., "Curriculum Learning," *ICML*, 2009.
- [8] W. Guss et al., "MineRL: A Large-Scale Dataset of Minecraft Demonstrations," *IJCAI*, 2019.
- [9] V. Mnih et al., "Human-Level Control Through Deep Reinforcement Learning," *Nature*, 2015.
- [10] O. Vinyals et al., "Grandmaster Level in StarCraft II Using Multi-agent Reinforcement Learning," *Nature*, 2019.
- [11] A. Ng, D. Harada, and S. Russell, "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping," *ICML*, 1999.
- [12] M. Johnson et al., "The Malmo Platform for Artificial Intelligence Experimentation," *IJCAI*, 2016.